LECTURE 23

# Dimensionality Reduction & Maximum-Margin Classifiers

Taming high-dimensional audio features and finding robust boundaries.

**75 minutes • PCA → SVM → pipeline**

Senior + first-year graduate ECE

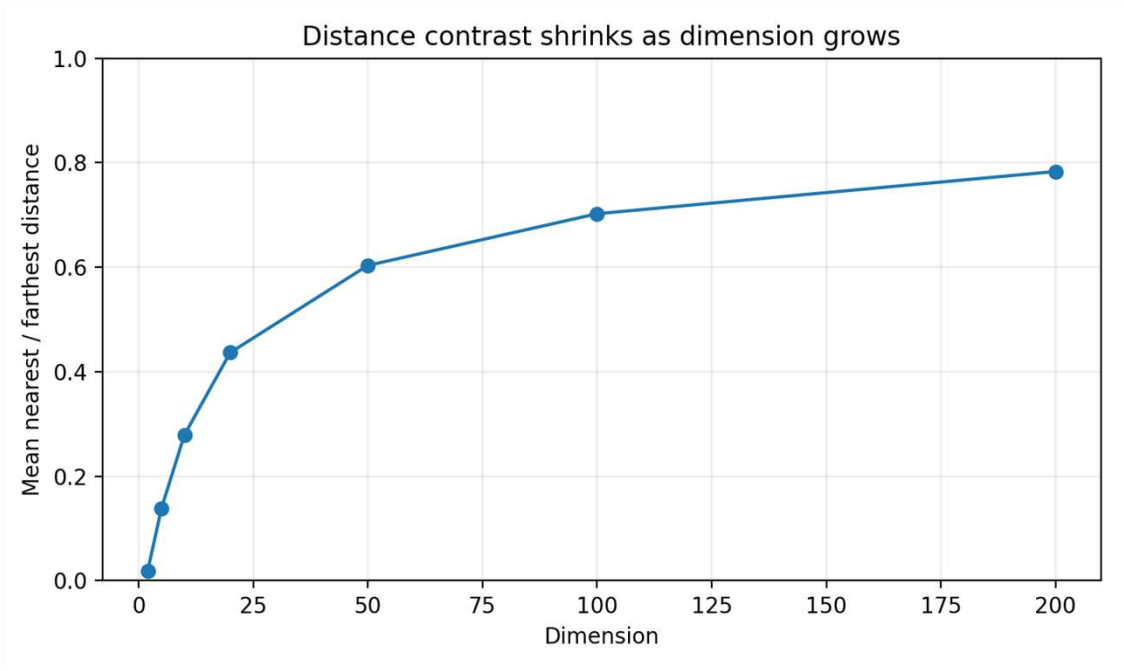
MOTIVATION

# The feature vector has exploded

- A 2 s clip with 20 ms frames can produce ~100 frames × 13 MFCCs — before adding energy, centroid, flux, roll-off, deltas, and statistics.
- High dimension is not automatically “more information”: many features are correlated, noisy, or redundant.
- Classification cost, sample complexity, visualization, and distance-based reasoning all become harder.

CURSE OF DIMENSIONALITY

# Distance loses contrast in high dimensions




**What changes?**

Nearest and farthest neighbors become less distinguishable. A distance metric can still work, but it needs more data and careful feature scaling / selection.

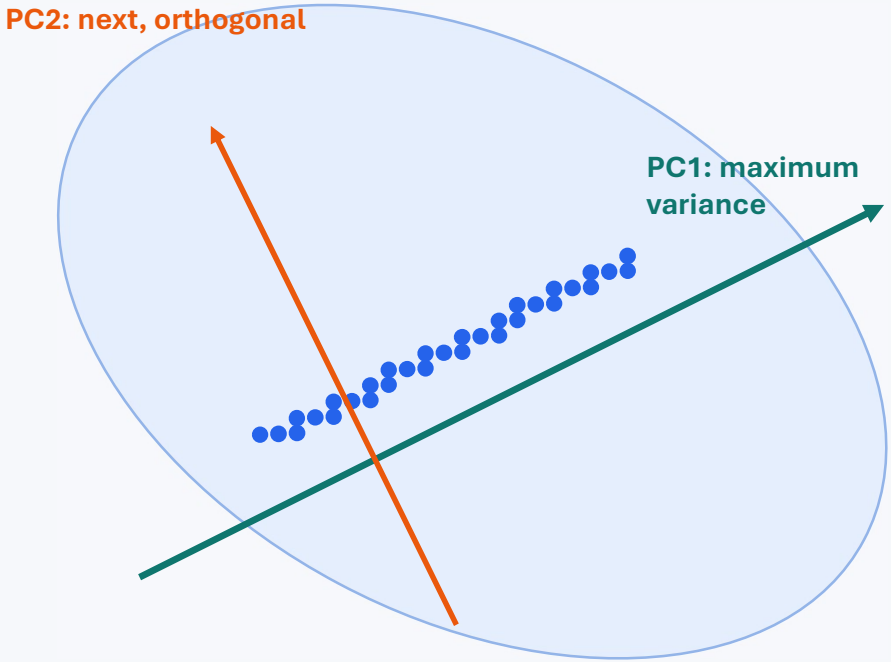
**Audio connection**

MFCCs, deltas, flux, energy and spectral shape often contain correlated evidence. PCA attacks redundancy, not “bad dimensions” one-by-one.

# PCA: the engineering goal

- Find orthogonal directions that capture as much variance in the standardized feature cloud as possible.
- Project each clip onto the first few directions to obtain a compact set of principal-component scores.
- Throw away low-variance directions only when that compression does not remove task-relevant structure.

# Geometry first: rotate, then drop directions




**Projection**

A data point becomes its coordinates along PC1, PC2, … . Keeping only PC1 is the best 1-D linear least-squares reconstruction subspace.

**Not feature selection**

PC1 is usually a mixture of centroid, roll-off, MFCCs, etc. PCA creates new axes; it does not merely pick original features.

PCA

# Step 0: standardize heterogeneous DSP features

$$z_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j}$$

- Centroid might be in Hz, RMS is amplitude-like, ZCR is a rate, and MFCCs have unrelated numerical scales.
- Without scaling, a feature with large numerical variance can dominate the covariance matrix.
- Fit mean and standard deviation on the training set only; apply the same transform to validation/test data.

**Exception**

If physical scale itself carries meaning and you intentionally want variance in those units to dominate, standardization is a modeling choice — not a universal law.

# From covariance to eigenvectors

$$\mathbf{C} = \frac{1}{N-1}\mathbf{X}^T\mathbf{X}$$

$$\mathbf{C}\mathbf{v}_k = \lambda_k \mathbf{v}_k$$

$$\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_D$$

**Eigenvector $v_k$**

A direction in feature space. Orthogonal covariance eigenvectors become the PCA axes.

**Eigenvalue $\lambda_k$**

Variance captured along that direction. Larger λ means the projected cloud spreads more.

**Scores**

For centered data, $z_k = Xv_k$. Each clip gets a coordinate along each principal direction.

WORKED EXAMPLE

# A 2-D PCA calculation by hand

Centered standardized feature samples give the covariance matrix

$$\mathbf{C} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} \quad \lambda_1 = 3,\ \mathbf{v}_1 = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 \\ 1 \end{bmatrix} \quad \lambda_2 = 1,\ \mathbf{v}_2 = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

**Interpretation**

PC1 is the “both features rise together” direction and captures 3/(3+1)=75% of total variance.

**DSP reading**

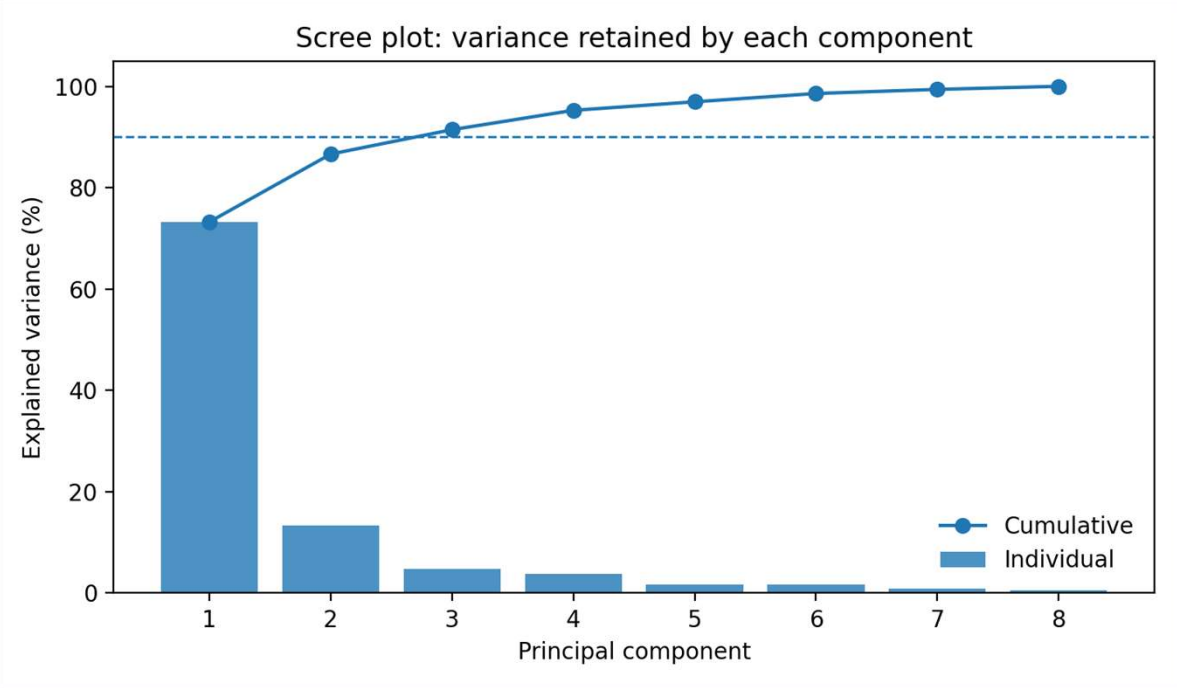
If the two features were centroid and roll-off, PC1 would represent their common spectral-brightness trend; PC2 measures their disagreement.

DSP CONNECTION

# PCA vs. the DCT in MFCC processing

- Both are orthogonal linear transforms that express a vector in a new basis.
- The DCT basis is fixed and data-independent; PCA learns its basis from the covariance structure of the training data.
- The DCT is chosen for useful signal-compaction properties on correlated sequences; PCA is optimal for variance capture on the observed distribution.
- MFCC pipeline: mel log energies → DCT → cepstral coefficients. PCA can then operate across a larger engineered feature vector.

# How many components? Use a scree plot + task validation




**Common rule**

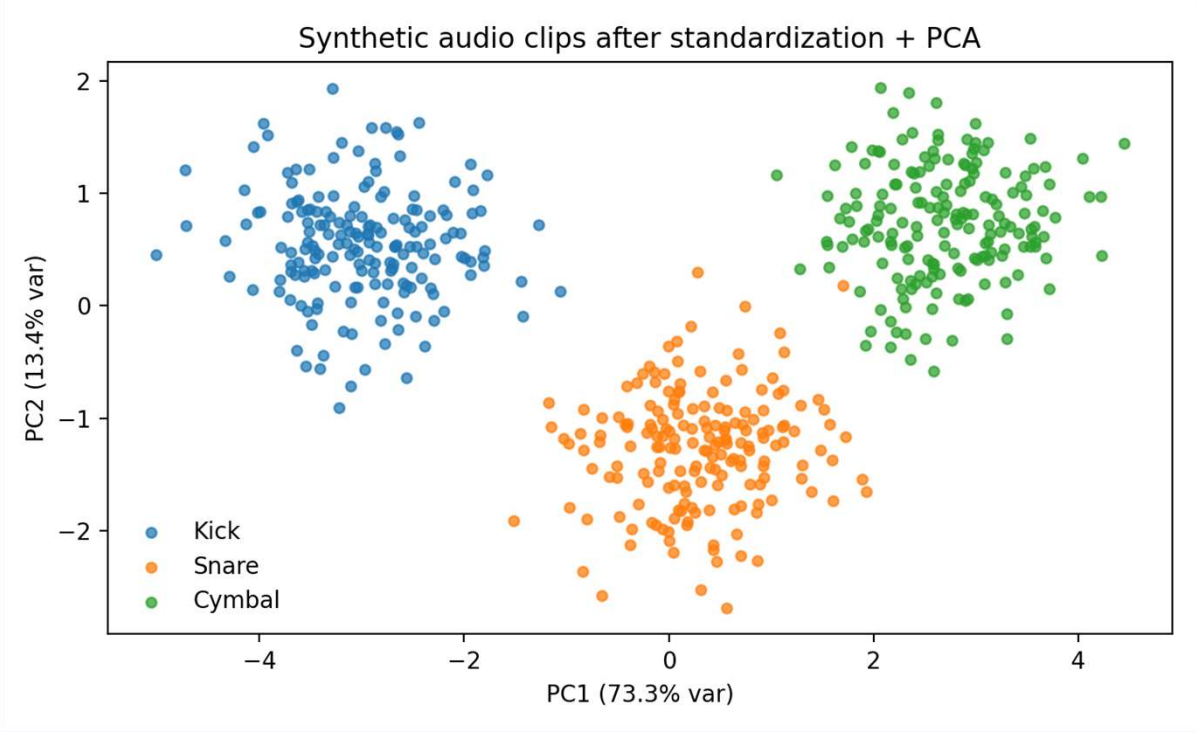
Choose the smallest k that retains a target fraction such as 90–95% variance — then verify downstream accuracy and stability.

**Do not say**

"95% variance means 95% classification accuracy." Explained variance is not a class-performance metric.

PCA IN PRACTICE

# What PCA reveals in our synthetic audio features




**Read the plot**

Classes separate substantially along combinations of features rather than along one raw feature.

**Important caveat**

This plot is synthetic and intentionally teachable. Real audio often has overlap from room acoustics, performer, microphone, gain, and recording context.

CONCEPT CHECK

# Pause & predict: PCA

**Question**

A low-variance feature direction cleanly separates two classes, while high-variance directions contain performer and microphone variation. What can PCA do wrong?

CONCEPT CHECK

# Pause & predict: PCA

**Question**

A low-variance feature direction cleanly separates two classes, while high-variance directions contain performer and microphone variation. What can PCA do wrong?

**Good answer**

If we keep only high-variance PCs, PCA may discard the low-variance discriminative direction. PCA optimizes reconstruction variance, not label separation.

Extension: Which method would explicitly use labels? → supervised feature selection, LDA, or a classifier trained on the full feature set.

IMPLEMENTATION

# scikit-learn: scaling → PCA

Leakage-safe preprocessing

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

prep = Pipeline([
    ("scale", StandardScaler()),
    ("pca", PCA(n_components=0.95)),
])
Z_train = prep.fit_transform(X_train)
Z_test  = prep.transform(X_test)
```

**Why a Pipeline?**

During cross-validation, each fold learns its own scaler and PCA basis from that fold's training samples only.

**Notebook**

Lecture23_PCA_SVM.ipynb includes scree plots, loadings, GridSearchCV, and a leakage-safe implementation.

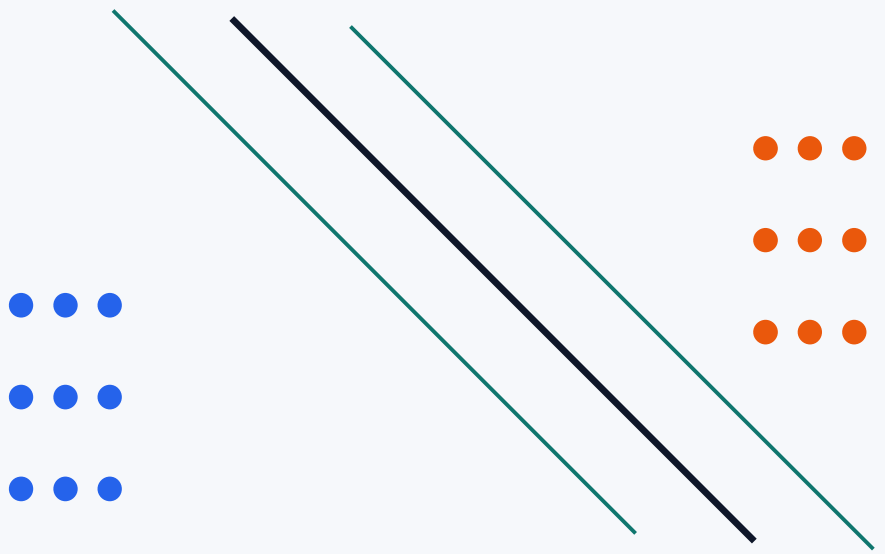
# Now classify: what boundary should we trust?

- A linear classifier can separate classes with a hyperplane — but infinitely many separating hyperplanes may exist.
- SVM asks for the separating boundary with the largest margin to the closest training examples.
- Those closest examples are the support vectors: they determine the boundary.

# Maximum-margin hyperplane

$$f(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + b$$

$$\text{margin} = \frac{2}{||\mathbf{w}||}$$



support vectors sit on / inside margin

# Hard margin becomes soft margin

$$\min_{\mathbf{w},b,\boldsymbol{\xi}} \ \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_i \xi_i$$

$$y_i\left(\mathbf{w}^T\mathbf{x}_i + b\right) \geq 1 - \xi_i,\ \xi_i \geq 0$$

**Small C**

Wider / smoother effective margin; tolerates more violations. More regularization, potentially more bias.

**Large C**

Penalizes violations strongly. Fits training labels more aggressively; can increase variance / overfit.

# Support vectors: why only a few points matter

- Points far from the margin can move slightly without changing the optimal boundary.
- Support vectors are the training examples at or inside the margin; they carry the geometric constraint.
- This gives SVMs a sparse geometric description of the boundary, even though kernel prediction may still be computationally significant.

# When a straight line is not enough: kernels

$$K(\mathbf{x}, \mathbf{x}') = \exp\left(-\gamma \|\mathbf{x} - \mathbf{x}'\|^2\right)$$

**Kernel trick**

Compute inner-product-like similarities corresponding to a higher-dimensional feature map without explicitly building that map.

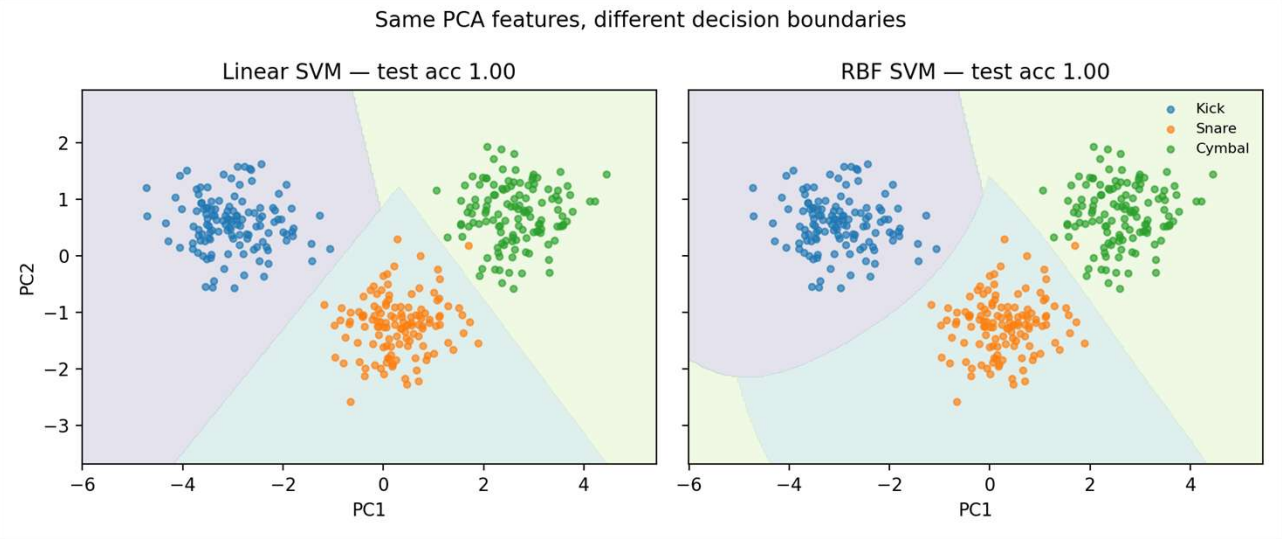
**γ small**

Broad influence; smoother decision function. Too small can underfit.

**γ large**

Very local influence; intricate boundary. Too large can memorize local details.

# Linear vs. RBF on the same PCA coordinates




**Do not infer too much from 2-D**

The full classifier may use more PCs. A pretty 2-D boundary is for intuition, not proof of generalization.

**Select by validation**

Tune C, γ, and PCA dimensionality with cross-validation — then report the final held-out test result once.

IMPLEMENTATION

# One pipeline: PCA → SVM

PCA + RBF SVM

```
from sklearn.svm import SVC

model = Pipeline([
  ("scale", StandardScaler()),
  ("pca", PCA(n_components=0.95)),
  ("svm", SVC(kernel="rbf", C=3, gamma="scale")),
])

model.fit(X_train, y_train)
print(model.score(X_test, y_test))
```

- Scale because both PCA and SVM depend on geometry.
- Keep transforms inside the pipeline so cross-validation is honest.
- Grid-search C, γ, and potentially PCA dimensionality on training folds only.

WRAP-UP

# Lecture 23 synthesis: what each step is buying you

**Feature engineering**

Inject DSP knowledge: summarize spectra, dynamics, timbre, energy.

**PCA**

Compress correlated geometry; improve visualization / conditioning; may reduce noise and compute.

**SVM**

Choose a robust margin-based decision surface; use kernels when justified.

**Raw audio → DSP features → scale → PCA? → classifier → validation**